SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I **C.BHUVANESH (192521108)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: c.bhuvanesh

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

Pseudocode: Network Throughput and Packet Loss

Given Values

- **Total data received = 8000 bits**
- **Time = 4 seconds**
- **Packets sent = 200**
- **Packets received = 180**

Algorithm

ALGORITHM Network_Performance

BEGIN

```
// Input values
total_data_received ← 8000
time ← 4
packets_sent ← 200
packets_received ← 180

// Validate input
IF time = 0 THEN
    DISPLAY "Error: Time cannot be zero."
    STOP
END IF

IF packets_sent = 0 THEN
    DISPLAY "Error: Total packets sent cannot be zero."
    STOP
END IF

// Calculate throughput
throughput ← total_data_received / time

// Calculate packet loss
packets_lost ← packets_sent - packets_received
packet_loss_percentage ← (packets_lost / packets_sent) × 100
```

```
// Display calculated results
DISPLAY "===== Network Performance ====="
DISPLAY "Total Data Received: ", total_data_received, " bits"
DISPLAY "Time: ", time, " seconds"
DISPLAY "Packets Sent: ", packets_sent
DISPLAY "Packets Received: ", packets_received
DISPLAY "Packets Lost: ", packets_lost
DISPLAY "Throughput: ", throughput, " bps"
DISPLAY "Packet Loss: ", packet_loss_percentage, "%"

// Classify network
IF throughput > 1500 AND packet_loss_percentage < 10 THEN
   classification ← "Efficient"
ELSE
   classification ← "Inefficient"
END IF

DISPLAY "Network Classification: ", classification

// Complete summary
DISPLAY "===== Performance Summary ====="

IF classification = "Efficient" THEN
   DISPLAY "The network has good throughput and low packet loss."
   DISPLAY "Overall Network Performance: Efficient"
ELSE
```

DISPLAY "The network does not satisfy the required throughput"

DISPLAY "and packet loss conditions."

DISPLAY "Overall Network Performance: Inefficient"

END IF

END

Calculation

1. Throughput

Throughput=TimeTotal Data Received =48000=2000 bps

2. Packets Lost

Packets Lost=200−180=20

3. Packet Loss Percentage

Packet Loss=20020×100=10%

Final Output

===== Network Performance =====

Total Data Received: 8000 bits

Time: 4 seconds

Packets Sent: 200

Packets Received: 180

Packets Lost: 20

Throughput: 2000 bps

Packet Loss: 10%

Network Classification: Inefficient

===== Performance Summary =====
The network does not satisfy the required throughput
and packet loss conditions.
Overall Network Performance: Inefficient

Reason: Although the throughput is 2000 bps > 1500 bps, the packet loss is 10%, and the condition requires packet loss to be less than 10%. Therefore, the network is classified as Inefficient.

2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

Algorithm

ALGORITHM Bandwidth_Monitoring

BEGIN

```
// Number of network links
INPUT number_of_links
```

```
IF number_of_links <= 0 THEN
    DISPLAY "Error: Number of links must be greater than zero."
    STOP
END IF

// Create an array to store bandwidth utilization
DECLARE bandwidth_usage[number_of_links]

// Continuously monitor all network links
WHILE TRUE DO

    DISPLAY "===== Bandwidth Monitoring ====="

    FOR i ← 0 TO number_of_links - 1 DO

        // Collect bandwidth utilization
        INPUT bandwidth_usage[i]

        // Validate bandwidth value
        IF bandwidth_usage[i] < 0 OR bandwidth_usage[i] > 100 THEN
            DISPLAY "Invalid bandwidth value for Link ", i + 1
            CONTINUE
        END IF

        // Display current utilization
        DISPLAY "Link ", i + 1, ": ",
```

```
            bandwidth_usage[i], "% utilized"

        // Check for overloaded link
        IF bandwidth_usage[i] > 80 THEN
            DISPLAY "WARNING: Link ", i + 1,
                " is overloaded."
            DISPLAY "Bandwidth utilization exceeds 80%."
            DISPLAY "Recommendation: Switch traffic to backup link."

        ELSE
            DISPLAY "Link ", i + 1,
                " is operating normally."
        END IF

    END FOR

    // Wait before next monitoring cycle
    WAIT 5 seconds

  END WHILE

END
```

Example

Suppose there are **4 network links**:

Link	Bandwidth Utilization	Status
Link 1	65%	Normal
Link 2	83%	⚠ Overloaded
Link 3	72%	Normal
Link 4	91%	⚠ Overloaded

The algorithm produces:

===== Bandwidth Monitoring =====

Link 1: 65% utilized
Link 1 is operating normally.

Link 2: 83% utilized
WARNING: Link 2 is overloaded.
Bandwidth utilization exceeds 80%.
Recommendation: Switch traffic to backup link.

Link 3: 72% utilized
Link 3 is operating normally.

Link 4: 91% utilized

WARNING: Link 4 is overloaded.

Bandwidth utilization exceeds 80%.

Recommendation: Switch traffic to backup link.

How the Algorithm Helps

1. **Continuous monitoring** – The `WHILE` loop repeatedly checks all network links.
2. **Stores utilization values** – The array `bandwidth_usage[]` keeps the current bandwidth usage of each link.
3. **Detects congestion early** – When utilization exceeds **80%**, the algorithm identifies the link as overloaded.
4. **Traffic redirection** – It recommends moving traffic to a **backup link**, reducing the load on the congested link.
5. **Prevents performance degradation** – Early detection can reduce packet loss, delay, and slow network performance.
6. **Efficient bandwidth management** – Network administrators can identify heavily used links and distribute traffic more effectively.

Conclusion

The algorithm provides a simple **real-time bandwidth monitoring system**. By continuously checking every link and triggering a warning when utilization exceeds **80%**, it helps administrators take corrective action before serious congestion occurs. This improves **network reliability, performance, and efficient use of available bandwidth**.

3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Pseudocode: Dynamic Best-Path Selection

Objective

The algorithm continuously monitors **bandwidth, delay, and utilization** of network links and selects the **best communication path** based on overall link quality instead of simply choosing the shortest path.

Link Quality Score

A simple scoring formula can be:

$$Score=(Wb\times NormalizedBandwidth)+(Wd\times InverseDelay)+(Wu\times InverseUtilization)$$

Where:

- **Higher bandwidth** → better score

- **Lower delay** → better score
- **Lower utilization** → better score
- `Wb`, `Wd`, and `Wu` are weights.

For example:

- Bandwidth weight = **0.4**
- Delay weight = **0.3**
- Utilization weight = **0.3**

Pseudocode

ALGORITHM Dynamic_Best_Path_Selection

BEGIN

```
// Six branch offices
number_of_branches ← 6

// Monitoring interval
monitoring_interval ← 10 seconds

// Threshold for overloaded links
overload_threshold ← 80%

// Weights for link quality calculation
bandwidth_weight ← 0.4
delay_weight ← 0.3
utilization_weight ← 0.3

// Arrays for storing link information
DECLARE bandwidth[]
DECLARE delay[]
DECLARE utilization[]
DECLARE link_status[]
DECLARE link_score[]

// List of possible paths from headquarters
DECLARE paths[]
```

```
WHILE TRUE DO

    DISPLAY "===== Network Monitoring Cycle ====="

    // Collect information for every network link
    FOR each link i in network_links DO

        bandwidth[i] ← COLLECT_BANDWIDTH(i)
        delay[i] ← COLLECT_DELAY(i)
        utilization[i] ← COLLECT_UTILIZATION(i)
        link_status[i] ← CHECK_LINK_STATUS(i)

        // Check whether link has failed
        IF link_status[i] = "FAILED" THEN

            link_score[i] ← 0

            DISPLAY "Link ", i, " has FAILED."
            GENERATE_ALERT(
                "Link failure detected on Link ", i
            )

            CONTINUE

        END IF

        // Check whether link is overloaded
        IF utilization[i] > overload_threshold THEN

            DISPLAY "WARNING: Link ", i, " is overloaded."

            GENERATE_ALERT(
                "High utilization on Link ", i
            )

            // Reduce score of overloaded link
            utilization_factor ← 0

        ELSE
```

```
        utilization_factor ←
            (100 - utilization[i]) / 100

    END IF

    // Normalize bandwidth
    bandwidth_factor ←
        bandwidth[i] / MAX_BANDWIDTH

    // Calculate delay factor
    delay_factor ←
        MIN_DELAY / delay[i]

    // Calculate link quality score
    link_score[i] ←
        (bandwidth_weight × bandwidth_factor)
        +
        (delay_weight × delay_factor)
        +
        (utilization_weight × utilization_factor)

END FOR


// Evaluate every possible path
FOR each path p in paths DO

    path_valid ← TRUE
    path_score ← 0

    FOR each link i in path p DO

        // Do not use failed links
        IF link_status[i] = "FAILED" THEN
            path_valid ← FALSE
            BREAK
        END IF

        // Do not prefer overloaded links
```

```
        IF utilization[i] > overload_threshold THEN
            path_valid ← FALSE
            BREAK
        END IF

        // Add link quality to path score
        path_score ← path_score + link_score[i]

    END FOR


    IF path_valid = TRUE THEN
        path_score[p] ← path_score
    ELSE
        path_score[p] ← 0
    END IF

END FOR


// Select path having highest score
best_path ← NULL
highest_score ← 0

FOR each path p in paths DO

    IF path_score[p] > highest_score THEN

        highest_score ← path_score[p]
        best_path ← p

    END IF

END FOR


// Check whether a valid path exists
IF best_path = NULL THEN

    DISPLAY "CRITICAL ALERT:"
```

```
        DISPLAY "No valid path available."
        GENERATE_ALERT(
            "All communication paths are unavailable."
        )

    ELSE

        DISPLAY "Best Path: ", best_path
        DISPLAY "Path Quality Score: ", highest_score

        // Check if current path has changed
        IF best_path ≠ current_path THEN

            DISPLAY "Routing path changed."
            DISPLAY "Switching traffic to new best path."

            current_path ← best_path

        END IF

    END IF


    // Wait before next monitoring cycle
    WAIT monitoring_interval

  END WHILE

END
```

Example

Suppose the headquarters has three possible paths to a branch:

Path	Bandwidth	Delay	Utilization	Status
Path A	100 Mbps	20 ms	85%	Overloaded

Path B	80 Mbps	15 ms	50%	Normal
Path C	120 Mbps	30 ms	40%	Normal

Although **Path A** has good bandwidth, its utilization is **85%**, exceeding the 80% threshold. Therefore, the algorithm avoids it and compares Paths B and C.

If Path C obtains the highest overall quality score, traffic is automatically routed through **Path C**.

If Path C later fails:

Path C → FAILED
↓
Generate Alert
↓
Recalculate available paths
↓
Select Path B
↓
Redirect Traffic

How the Algorithm Improves the Network

1. Improved Network Reliability

The algorithm continuously monitors every link. If a link fails, it detects the failure and selects another available path.

2. Better Routing Efficiency

Instead of choosing a path only because it is geographically short, the algorithm considers:

- **Bandwidth**
- **Delay**
- **Utilization**
- **Link availability**

Therefore, a slightly longer path may be selected if it provides better performance.

3. Fault Tolerance

When a link fails or becomes overloaded, traffic can be moved to an alternate path. This prevents a single link failure from interrupting communication.

4. Dynamic Adaptation

Network conditions change throughout the day. Because the algorithm periodically collects new measurements, the best path can change automatically.

For example:

Morning:
Path A → Best

Afternoon:
Path B → Best

Evening:
Path C → Best

The routing decision adapts to the current network conditions.

5. Early Congestion Detection

When utilization exceeds **80%**, the algorithm generates an alert and avoids the overloaded link where an alternative exists. This helps reduce:

- Packet loss
- Network delay
- Congestion
- Poor application performance

6. Centralized Administrator Alerts

The administrator receives alerts whenever:

- A link fails
- Utilization exceeds 80%
- No valid path is available
- The routing path changes

Conclusion

This algorithm provides **dynamic path selection based on overall link quality rather than shortest distance**. By continuously monitoring bandwidth, delay, utilization, and link status, it can select the best available path and automatically switch to an alternate route during failures or congestion.

Thus, it improves **routing efficiency, reliability, fault tolerance, and network performance** while adapting to changing conditions throughout the day.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases;	Handles most cases;	Handles basic cases only	Incomplete or inefficient

		optimized approach	reasonable efficiency		